

 Se former autrement HONORIS UNITED UNIVERSITIES	Année Universitaire : 2026-2027 Applications Web Distribuées UP_WEB
Implémentation « Eureka server »	

Mise en situation

- Dans le cas d'une application qui répond à une montée en charge et dans le cas de plusieurs instances de chaque Microservice, il est nécessaire d'avoir un **registre de toutes les instances disponibles** afin de distribuer la charge entre celles-ci.
 - **Eureka** de Netflix est une solution qui permet d'enregistrer les instances des Microservices et contient toutes les informations, sur quelle adresse IP et quel port les microservices client sont exécutés.
- L'appel d'un Microservice s'effectue à travers la liste des instances qu'Eureka expose via une **API REST**.
- Comme Eureka expose une API REST, des microservices écrits dans **des langages différents** (Java, JavaScript, Python) peuvent tous s'y enregistrer. C'est le cas de notre projet.

Objectifs

- Créer un **serveur de découverte « Eureka »**.
- Enregistrer les microservices Spring Boot **Candidat** et **Job** comme clients Eureka.
- Instancier un microservice plusieurs fois et observer ses instances dans Eureka.
- Enregistrer des microservices **Node.js** et **Python** dans Eureka (travail en équipe).

Architecture du projet

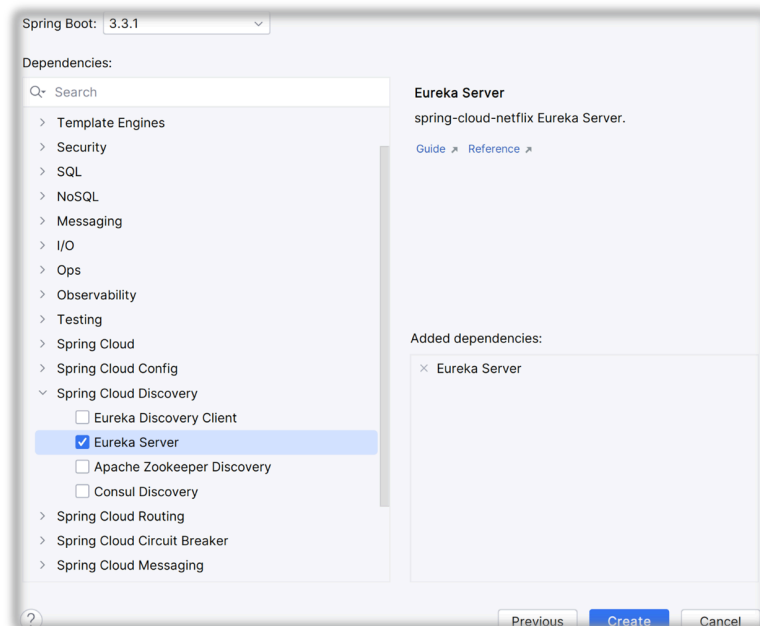
Tous les microservices s'enregistrent auprès du même serveur Eureka, puis se retrouvent grâce à leur **nom** (`spring.application.name`) au lieu d'une adresse écrite en dur.

Microservice	Technologie	Port	Nom dans Eureka	Réalisé
Eureka Server	Spring Boot	8761	–	Partie 1 (en séance)
Candidat	Spring Boot + H2	8081	CANDIDAT	Partie 2 (en séance)
Job	Spring Boot + MySQL	8082	JOB	Partie 2 (en séance)
Meeting	Node.js / Express	8083	MEETING	Partie 3 (en équipe)
Notification	Python / FastAPI	8084	NOTIFICATION	Partie 3 (en équipe)

Partie 1 : Mise en place de serveur « Eureka »

Etape 1 : Création d'une application Spring Eureka Server

Créer une application **Spring** et ajouter la dépendance « **Eureka Server** ».



Remarque : choisir une version de Spring Boot supportée par Spring Cloud (par exemple Spring Boot 3.5.x avec Spring Cloud 2025.0.x). Le générateur ajoute automatiquement la propriété `spring-cloud.version` et la section `dependencyManagement` au fichier **POM.xml**.

Vérifier si la dépendance ajoutée apparaît comme suit dans le fichier **POM.xml**.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

Etape 2 : Spring Boot Starter Application

Dans la classe **EurekaApplication.java**, en haut de la classe ajouter l'annotation suivante :

@EnableEurekaServer : utilisée pour que l'application Spring Boot se comporte comme un serveur Eureka.

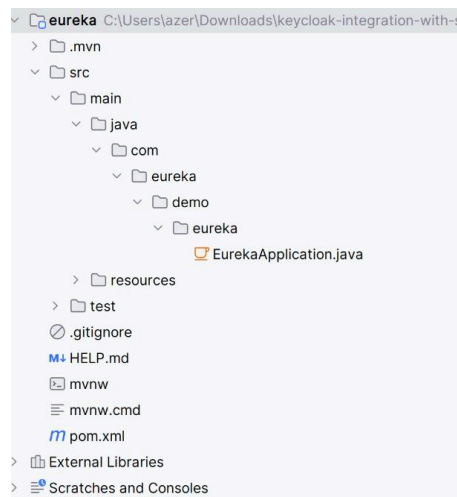
```

1 package com.example.demo;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
6
7 @SpringBootApplication
8 @EnableEurekaServer
9 public class EurekaApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(EurekaApplication.class, args);
13     }
14
15 }

```

Etape 3 : Application Properties File

Dans le fichier **application.properties** situé à l'adresse `src / main / resources`, ajouter la liste des propriétés ci-dessous.



```

1 spring.application.name=eureka
2 server.port=8761
3 eureka.client.register-with-eureka=false
4 eureka.client.fetch-registry=false

```

spring.application.name donne un nom unique pour l'application.

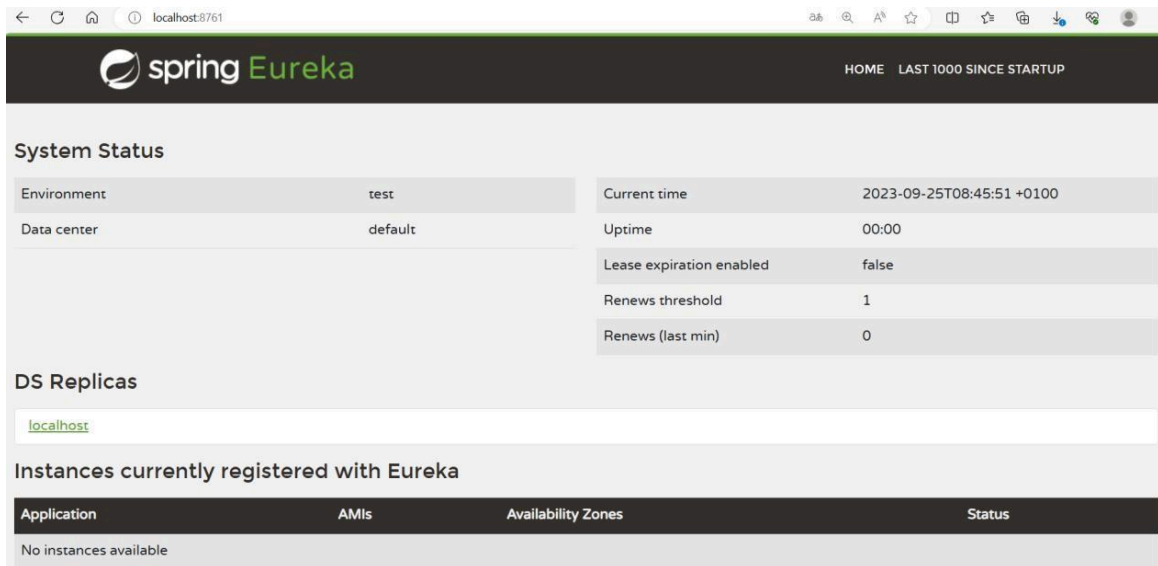
server.port définit le port (8761) utilisé par défaut pour le serveur Eureka.

eureka.client.fetch-registry : indique qu'il ne faut pas mettre en cache localement la liste des clients enregistrés, puisqu'on est le serveur.

eureka.client.register-with-eureka : ne pas s'auto-déclarer sur Eureka, puisqu'on est le serveur.

Etape 4 : Exécution du serveur **Port : 8761**

Le résultat de l'exécution est comme le montre la figure ci-dessous. Aucune application n'est encore enregistrée (« *No instances available* »).



Aucune application : aucun microservice n'est encore enregistré.

Partie 2 : découverte de Microservices (Eureka clients)

La découverte de services est l'un des principes clés de l'architecture Microservices. La configuration manuelle de chaque client est difficile à faire.

Eureka est le serveur Netflix de découverte de services et de clients qui peut être configuré et déployé pour être hautement disponible, chaque serveur répliquant l'état des services enregistrés aux autres.

Etape 1 : Intégration du client Eureka

- Dans le fichier **pom.xml** du microservice **Candidat**, ajouter la dépendance suivante :

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

- Si le projet n'a pas été créé avec une dépendance Spring Cloud, ajouter aussi la version de Spring Cloud et sa nomenclature (BOM) :

```
<properties>
  <spring-cloud.version>2025.0.0</spring-cloud.version>
</properties>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Etape 2 : S'inscrire à Eureka

Lorsqu'un client s'enregistre auprès d'Eureka, il fournit des métadonnées sur lui-même, telles que l'hôte et le port, l'URL, la page d'accueil, etc. Eureka reçoit ensuite des messages de renouvellement (*heartbeat*) de chaque instance toutes les 30 secondes. Si ces messages s'arrêtent (90 secondes par défaut), l'instance est supprimée du registre.

1. Dans le fichier **CandidatApplication.java**, ajouter l'annotation :

@EnableDiscoveryClient : sert à annoter que l'application sera transformée en Eureka Client.

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
@EnableDiscoveryClient
public class CandidatApplication {

    public static void main(String[] args) {

        SpringApplication.run(CandidatApplication.class, args);
    }
}
```

Remarque : avec les versions récentes de Spring Cloud, la présence de la dépendance suffit et l'annotation est facultative. On la garde pour rendre le rôle de l'application explicite.

2. Dans le fichier **application.properties** (application Candidat), ajouter les lignes suivantes sachant que :

eureka.client.service-url.defaultZone fournit l'URL du serveur Eureka à tout client n'exprimant pas de préférence (il s'agit d'un paramètre utile par défaut).

```
# Service
spring.application.name=candidat
server.port=8081

# eureka registration
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
```

3. Refaire les mêmes étapes pour le microservice **Job** avec `spring.application.name=job` et `server.port=8082`.

Etape 3 : Authentification avec le serveur Eureka

L'authentification HTTP de base sera automatiquement ajoutée à votre client Eureka si l'une des URL **eureka.client.service-url.defaultZone** contient des informations d'identification (exemple : `http://user:password@localhost:8761/eureka`).

Etape 4 : Lancer plusieurs instances d'un microservice

Pour simuler une montée en charge, lancer une deuxième instance de **Candidat** sur un autre port :

- **IntelliJ IDEA** : *Run* → *Edit Configurations*, dupliquer la configuration de **CandidatApplication** (*Copy Configuration*), puis dans *Modify options* → *Add VM options* saisir `-Dserver.port=8091`.
- **Ligne de commande** : `mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8091`

Remarque : Candidat utilise une base H2 en mémoire, chaque instance a donc ses propres données.

Le résultat de l'exécution est le suivant (URL : `localhost:8761`) :

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
CANDIDAT	n/a (2)	(2)	UP (2) - MON-PC:candidat:8081 , MON-PC:candidat:8091
JOB	n/a (1)	(1)	UP (1) - MON-PC:job:8082

Résultat attendu : deux instances de CANDIDAT et une instance de JOB (le nom de machine dépend de votre poste).

Partie 3 : Clients Eureka Node.js et Python (travail en équipe)

Les microservices **Meeting** (Node.js / Express) et **Notification** (Python / FastAPI) exposent pour l'instant un seul endpoint *hello*. Chaque équipe doit les enregistrer dans le **même serveur Eureka** que Candidat et Job, en utilisant une bibliothèque cliente adaptée à chaque langage.

Tâche 1 : Microservice Meeting (Node.js / Express)

1. Ajouter la dépendance **eureka-js-client** au projet (`npm install eureka-js-client`).
2. Enregistrer le service sous le nom **MEETING** au démarrage de l'application.
3. Déclarer l'hôte, l'adresse IP, le port (**8083**) et le *vipAddress* de l'instance.
4. Désenregistrer proprement le service à l'arrêt de l'application (signal SIGINT).
5. Lire le port depuis la variable d'environnement `PORT` afin de pouvoir lancer plusieurs instances.

Indice : la propriété *dataCenterInfo* doit contenir `@class : 'com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo'` et `name : 'MyOwn'`, et le *servicePath* du serveur Eureka est `/eureka/apps/`.

Tâche 2 : Microservice Notification (Python / FastAPI)

1. Ajouter **py-eureka-client** dans le fichier `requirements.txt`.
2. Enregistrer le service sous le nom **NOTIFICATION** au démarrage de FastAPI (événement *lifespan*).
3. Déclarer le port de l'instance (**8084**) et l'URL du serveur `http://localhost:8761/eureka`.
4. Arrêter le client Eureka à l'arrêt de l'application.
5. Lire le port depuis une variable d'environnement afin de pouvoir lancer plusieurs instances.

Indice : le port déclaré à Eureka doit être **le même** que celui passé à Uvicorn :
`python -m uvicorn app.main:app --port 8084`

Tâche 3 : Vérification

Chaque microservice doit :

- être enregistré automatiquement dans Eureka ;
- être visible dans le dashboard (*http://localhost:8761*) ;
- pouvoir être exécuté sur plusieurs instances (ports différents) ;
- continuer à répondre sur son endpoint *hello* :
GET *http://localhost:8083/api/meetings/hello*
GET *http://localhost:8084/api/notifications/hello*

Bonus

- Ajouter un endpoint GET */health* à Meeting et Notification et le déclarer comme *healthCheckUrl* / *statusPageUrl* dans Eureka.
- Depuis Meeting ou Notification, **découvrir** l'adresse de **CANDIDAT** via Eureka (sans URL écrite en dur) et appeler GET */api/candidates/{id}*.

Rendu attendu (par équipe)

- Le projet **Eureka Server** fonctionnel.
- Les microservices **Meeting** (Node.js) et **Notification** (Python) configurés comme **Eureka Clients**.
- Les quatre services CANDIDAT, JOB, MEETING et NOTIFICATION visibles en même temps dans le dashboard.
- **Au moins deux instances** de MEETING ou de NOTIFICATION visibles (ports différents).
- Une capture d'écran du dashboard Eureka dans le README de l'équipe.
- Le projet poussé sur **GitHub**, avec les noms des membres de l'équipe dans le README.

Conseil : démarrer d'abord le serveur Eureka, puis Candidat et Job, et enfin Meeting et Notification.
Un service peut mettre jusqu'à 30 secondes avant d'apparaître dans le dashboard.